



APIs RESTful

Construção, Documentação e Boas Práticas



Autenticação e Autorização

JWT e Middlewares de Proteção

O que é uma API REST?

REST (Representational State Transfer) é um estilo de arquitetura para comunicação entre sistemas via HTTP.



Cliente / Servidor

Separação de responsabilidades: quem pede (cliente) e quem responde (servidor) são independentes.



Stateless

Cada requisição é independente. O servidor não guarda estado entre chamadas — tudo que ele precisa saber vem na própria requisição.



Interface Uniforme

Endpoints previsíveis, verbos HTTP padronizados e respostas JSON consistentes.



Recursos

Tudo é um recurso identificado por uma URL: /usuarios, /produtos, /pedidos.

Verbos HTTP

GET

Buscar dados. Não altera nada no servidor.

GET /filmes

POST

Criar um novo recurso.

POST /filmes

PUT

Substituir um recurso por completo.

PUT /filmes/1

PATCH

Atualizar campos específicos de um recurso.

PATCH /filmes/1

DELETE

Remover um recurso.

DELETE /filmes/1

Status Codes

2xx — Sucesso

-  200 OK — retorno padrão
-  201 Created — recurso criado
-  204 No Content — sem body (DELETE)

3xx — Redirecionamento

-  301 Moved — mudou de URL
-  304 Not Modified — use o cache

4xx — Erro do cliente

-  400 Bad Request — body inválido
-  401 Unauthorized — sem autenticação
-  403 Forbidden — sem permissão
-  404 Not Found — não existe

5xx — Erro do servidor

-  500 Internal Server Error
-  503 Service Unavailable

Estrutura de Endpoints — Boas Práticas



GET /filmes

Use substantivos no plural para coleções



GET /getFilmes

Não coloque verbos na URL



GET /filmes/42

Use o ID para acessar um recurso específico



GET /filmes/42/atores

Use hierarquia para expressar relações



GET /filme

Prefira plural para coleções



GET /filmes?genero=Ação

Use query string para filtros e buscas

Boas Práticas — Requisição e Resposta

Requisição

```
// Headers essenciais
Content-Type: application/json
Authorization: Bearer <token>

// Body bem estruturado (POST/PUT)
{
  "titulo": "Matrix",
  "ano": 1999,
  "nota": 8.7
}
```

Resposta padronizada

```
// Sucesso
{
  "sucesso": true,
  "dados": { ... }
}

// Erro
{
  "sucesso": false,
  "erro": "Mensagem clara"
}
```

Regras de ouro:

- Sempre retorne JSON com Content-Type: application/json
- Use o status code correto — não retorne 200 para erros
- Nunca exponha stack trace ou detalhes internos

Versionamento e Idempotência

Versionamento

Permite evoluir a API sem quebrar clientes que já a usam.

/api/v1/filmes ← versão atual
/api/v2/filmes ← nova versão

Nunca quebre uma versão publicada — crie uma nova.

Idempotência

Uma operação é idempotente se executá-la N vezes tem o mesmo efeito que executá-la uma vez.

✓ GET, PUT, DELETE

Idempotentes

✗ POST

Não idempotente

Verbo	Operação	Idempotente? (repetir não muda o resultado)	Seguro? (não altera dados no servidor)
GET	Leitura	✓ Sim	✓ Sim
POST	Criação	✗ Não	✗ Não
PUT	Substituição	✓ Sim	✗ Não
DELETE	Remoção	✓ Sim	✗ Não

Documentação com Swagger / OpenAPI

Uma API sem documentação é como um produto sem manual — ninguém sabe como usar.



OpenAPI

Padrão aberto para descrever APIs REST em JSON ou YAML. Define rotas, parâmetros, schemas e respostas.



Swagger UI

Interface visual gerada a partir da spec OpenAPI. Permite testar endpoints direto no navegador.



@fastify/swagger

Plugin oficial do Fastify que gera a spec automaticamente a partir dos schemas das rotas.

Fastify + Swagger — Instalação e Configuração

```
npm install @fastify/swagger @fastify/swagger-ui
```

```
// server.js
const fastify = require('fastify')({ logger: true });

// 1. Gera a spec OpenAPI automaticamente
fastify.register(require('@fastify/swagger'), {
  openapi: {
    info: { title: 'API de Filmes', version: '1.0.0' },
  },
});

// 2. Interface visual em /docs
fastify.register(require('@fastify/swagger-ui'), {
  routePrefix: '/docs',
});

fastify.register(require('./src/routes/filmeRoutes'));
fastify.listen({ port: 3000 });
```

Acesse <http://localhost:3000/docs> para ver a documentação visual gerada automaticamente.

Anotando Rotas com Schema

```
// src/routes/filmeRoutes.js

fastify.get('/filmes', {
  schema: {
    description: 'Lista todos os filmes',
    tags: ['Filmes'],
    querystring: {
      type: 'object',
      properties: {
        genero: { type: 'string', description: 'Filtrar por gênero' },
      },
    },
    response: {
      200: {
        type: 'object',
        properties: {
          total: { type: 'number' },
          filmes: { type: 'array', items: { type: 'object' } },
        },
      },
    },
  },
})
```

O schema serve para duas coisas: gerar a documentação automaticamente E validar os dados da requisição.

Swagger UI e Boas Práticas de Documentação

O que o Swagger UI exhibe:

- Todas as rotas agrupadas por tag
 - Parâmetros esperados (path, query, body)
 - Exemplos de resposta por status code
 - Botão Try it out — testa a rota direto no navegador
-

Boas práticas:

- ✓ Descreva todos os possíveis status codes de erro (400, 404, 500)
- ✓ Use tags para agrupar rotas por recurso (Filmes, Usuários...)
- ✓ Mantenha a documentação atualizada — schema desatualizado engana
- ⚠ Não documente só o caminho feliz — erros também precisam de docs

Resumo — APIs RESTful



REST

Stateless, cliente/servidor, recursos via HTTP



Verbos HTTP

GET · POST · PUT · PATCH · DELETE



Status Codes

2xx sucesso · 4xx cliente · 5xx servidor



Endpoints

Substantivos, plural, hierarquia, sem verbos



Documentação

OpenAPI + Swagger UI + @fastify/swagger



Autenticação e Autorização

JWT e Middlewares de Proteção

Fluxo completo de login e controle de acesso com Fastify

Autenticação vs Autorização



Autenticação

Quem é você?

Verifica a identidade do usuário.

É o processo de login — confirmar que você é quem diz ser.

Exemplos:

- Email + senha no login
- Token JWT enviado na requisição
- Biometria, Google OAuth



Autorização

O que você pode fazer?

Define o que o usuário autenticado tem permissão de acessar ou executar.

Exemplos:

- Admin pode deletar usuários
- Usuário comum só lê os próprios dados
- Rota /admin bloqueada para não-admins

Sessão vs Token (JWT)

Sessão (tradicional)

- Servidor guarda a sessão em memória ou banco
- Cliente recebe um session ID via cookie
- Toda requisição consulta o servidor para validar
- Difícil de escalar horizontalmente
- Funciona bem para aplicações web monolíticas

Token JWT (stateless)

- Servidor não guarda nada — token é autocontido
- Cliente envia o token no header Authorization
- Validação feita assinando/verificando o token localmente
- Escala muito bem — qualquer servidor valida
- Ideal para APIs REST e microserviços

O que é JWT?

JWT (JSON Web Token) é uma string dividida em 3 partes separadas por ponto (.)

eyJhbGciOiJIUzI1NiJ9 . eyJ1c2VySWQiOiIxMjMifQ . SflKxwRJSMeKKF2QT4fw

HEADER

Algoritmo

1. Header

Diz qual algoritmo foi usado para assinar o token.

```
{ "alg": "HS256", "typ": "JWT" }
```

Base64 decodificável — não é segredo

PAYLOAD

Dados do usuário

2. Payload

Carrega os dados do usuário (chamados de claims).

```
{ "userId": "123",  
  "role": "admin",  
  "exp": 1710000000 }
```

Base64 decodificável — nunca coloque senha aqui!

SIGNATURE

Verificação

3. Signature

Prova que o token não foi adulterado. Gerada com a chave secreta do servidor.

```
HMACSHA256(  
  base64(header) + base64(payload),  
  JWT_SECRET  
)
```

Só o servidor conhece o JWT_SECRET

Fluxo Completo de Autenticação

1

Login

Cliente envia email + senha

2

Valida senha

Servidor verifica com bcrypt

3

Gera JWT

Servidor cria e assina o token

4

Retorna token

Cliente recebe e armazena o JWT

5

Req. protegida

Cliente envia JWT no header

6

Valida token

Middleware verifica assinatura +
expiração

7

Acesso liberado

Rota executa e retorna os dados

Instalação e Configuração — @fastify/jwt

```
npm install @fastify/jwt bcrypt
```

```
// server.js
require('dotenv').config();
const fastify = require('fastify')({ logger: true });

// Registra o plugin JWT com a chave secreta
fastify.register(require('@fastify/jwt'), {
  secret: process.env.JWT_SECRET, // nunca coloque no código — use .env
});

fastify.register(require('./src/routes/authRoutes'));
fastify.register(require('./src/routes/filmeRoutes'));

fastify.listen({ port: 3000 });
```

```
# .env
JWT_SECRET=minha_chave_secreta_muito_longa_e_segura
PORT=3000
```

Rota POST /login

```
const bcrypt = require('bcrypt');

async function login(request, reply) {
  const { email, senha } = request.body;

  // 1. Busca o usuário no banco pelo email
  const usuario = await db.usuarios.findOneAsync({ email });
  if (!usuario) {
    return reply.code(401).send({ erro: 'Email ou senha inválidos' });
  }

  // 2. Compara a senha com o hash salvo no banco
  const valida = await bcrypt.compare(senha, usuario.senhaHash);
  if (!valida) {
    return reply.code(401).send({ erro: 'Email ou senha inválidos' });
  }

  // 3. Gera o token JWT com os dados do usuário
  const token = await reply.jwtSign(
    { userId: usuario._id, role: usuario.role },
    { expiresIn: '2h' }
  );
}
```

O que são Middlewares?

Middlewares são funções executadas entre o recebimento da requisição e a execução do handler da rota.



No Fastify, middlewares são implementados via Hooks — especialmente o `onRequest`:

```
fastify.addHook('onRequest', async (request, reply) => {  
  // executado antes de qualquer rota  
  // se lançar um erro, a requisição é interrompida aqui  
});
```

Diferença: hooks `onRequest` rodam antes do parsing do body. Para validar auth, use `preHandler`.

Criando o Middleware de Autenticação

```
// src/middlewares/autenticar.js

async function autenticar(request, reply) {
  try {
    // Verifica a assinatura e a expiração do token
    await request.jwtVerify();
    // Depois disso: request.user = { userId: 1, role: 'admin' }
  } catch (err) {
    return reply.code(401).send({
      erro: 'Token inválido ou expirado',
    });
  }
}
```

```
module.exports = { autenticar };
```

Como usar nas rotas:

```
const { autenticar } = require('../middlewares/autenticar');

// Passa o middleware no preHandler — roda antes do handler
fastify.get('/filmes', { preHandler: autenticar }, listarFilmes);
```

Payload do Token e Expiração

O que guardar no payload:

✓	ID do usuário (userId)	<i>identifica sem precisar de nome</i>
✓	Papel / role (admin, user)	<i>para controle de acesso</i>
✓	Expiração (exp)	<i>gerado automaticamente pelo plugin</i>
✗	Senha ou hash da senha	<i>NUNCA — o payload é decodificável</i>
✗	Dados sensíveis (CPF, cartão)	<i>NUNCA — JWT não é criptografado</i>

Expiração — expiresIn:

```
reply.jwtSign(payload, { expiresIn: '2h' }) // 2 horas — recomendado
reply.jwtSign(payload, { expiresIn: '7d' }) // 7 dias — refresh token
reply.jwtSign(payload, { expiresIn: '15m' }) // 15 min — alta segurança
```

Níveis de Acesso — Roles

Middleware base — verifica o token:

```
async function autenticar(request, reply) {  
  try {  
    await request.jwtVerify();  
  } catch {  
    reply.code(401).send({ erro: 'Não autenticado' });  
  }  
}
```

Middleware de role — verifica a permissão:

```
function exigirRole(role) {  
  return async (req, reply) => {  
    await autenticar(req, reply);  
    if (req.user.role !== role)  
      reply.code(403).send({ erro: 'Acesso negado' });  
  };  
}
```

Aplicando nas rotas:

Armazenando Senhas com bcrypt

Nunca salve senhas em texto puro. Use bcrypt para gerar um hash seguro e irreversível.

Cadastro — gerar o hash:

```
async function cadastrar(request, reply) {  
  const { email, senha } = request.body;  
  const hash = await bcrypt.hash(senha, 10); // 10 = salt rounds  
  await db.usuarios.insertAsync({ email, senha: hash });  
  return reply.code(201).send({ mensagem: 'Usuário criado!' });  
}
```

Login — comparar a senha com o hash:

```
async function login(request, reply) {  
  const { email, senha } = request.body;  
  const usuario = await db.usuarios.findOneAsync({ email });  
  
  if (!usuario) return reply.code(401).send({ erro: 'Inválido' });  
  
  const valida = await bcrypt.compare(senha, usuario.senha);  
  if (!valida) return reply.code(401).send({ erro: 'Inválido' });  
}
```


Fluxo Completo — Registro, Login e Rota Protegida

1

POST /registro

Recebe email+senha → gera hash bcrypt → salva no banco

2

POST /login

Valida email → compara hash bcrypt → gera JWT → retorna

3

GET /filmes (protegida)

Recebe JWT no header → middleware valida → retorna dados

O token JWT é o elo entre o login e o acesso às rotas protegidas.

Boas Práticas de Segurança



Use HTTPS em produção

Sem HTTPS, o token trafega em texto puro e pode ser interceptado.



JWT_SECRET longo e aleatório

Mínimo 32 caracteres. Nunca commite no Git. Guarde no .env ou em variáveis de ambiente.



Defina expiração nos tokens

Tokens sem expiração são riscos permanentes. Use 2h para access tokens.



Nunca exponha dados sensíveis

Stack traces, senhas ou dados internos não devem aparecer na resposta de erro.



Rate limiting no login

Limite tentativas de login para evitar ataques de força bruta.

Resumo Geral



APIs RESTful

- Verbos HTTP: GET · POST · PUT · PATCH · DELETE
- Status codes: 2xx · 4xx · 5xx
- Endpoints com substantivos e hierarquia
- Respostas padronizadas em JSON
- Documentação com Swagger / OpenAPI
- @fastify/swagger gera a doc automaticamente



JWT e Middlewares

- Autenticação (quem?) vs Autorização (pode?)
- JWT: Header · Payload · Signature
- Login com bcrypt + geração do token
- Middleware autenticar via preHandler
- Roles: controle de acesso por nível
- Boas práticas: HTTPS, secret, expiração